

OPTIMASI JALUR NAVIGASI ROBOT MOBILE BERBASIS ALGORITMA A-STAR PADA PETA GRID

Pardede Jasman ¹, Subagdja Tegar ²

ABSTRACT:

Perencanaan jalur adalah salah satu teknologi kunci dalam navigasi robot *mobile*, dengan tujuan utama untuk menentukan jalur optimal atau *suboptimal* yang bebas tabrakan dari posisi awal menuju posisi tujuan [1]. Algoritma A-Star (A^*) dikenal luas karena efisiensinya, namun masih memiliki keterbatasan dalam hal kecepatan komputasi, jumlah simpul pencarian, dan banyaknya belokan [2]. Penelitian ini mengusulkan optimasi terhadap algoritma A^* dengan menggabungkan beberapa metode seperti *Jump Point Search* [3], *Bidirectional Search* [4], *Guide Line* [5], *Barrier Raster Coefficient* [6], *Turn Penalty Function* [6], dan *Path Polyline Optimization* [7]. Hasil simulasi menunjukkan bahwa algoritma A^* setelah dioptimasi mampu mengurangi jumlah simpul sebesar 85,12%, jumlah belokan sebesar 52,06%, serta meningkatkan efisiensi waktu pencarian hingga 61,97% dibandingkan A^* tradisional. Panjang lintasan hanya berubah sedikit, yakni sebesar 0,82%. Temuan ini membuktikan bahwa efisiensi dan kelancaran pergerakan robot meningkat tanpa mengorbankan kualitas jalur.

Keyword : Algoritma A-Star, Perencanaan Jalur, Navigasi Robot, Optimasi Algoritma, Efisiensi Komputasi, Peta Grid.

1. Pendahuluan

Algoritma A^* telah menjadi salah satu algoritma perencanaan jalur global yang paling banyak digunakan karena kemudahan implementasinya, efisiensi pencarian yang tinggi, dan penerapan yang luas [2].

Namun, algoritma A^* menghadapi masalah lain yang sering muncul antara lain adalah banyaknya belokan tajam dalam jalur yang dihasilkan, serta durasi transportasi yang tidak optimal [8]. Kelemahan lain dari algoritma A^* tradisional termasuk kecepatan komputasi yang

lambat, jumlah simpul pencarian yang tinggi, yang semuanya dapat memperpanjang waktu komputasi dan menghasilkan jalur yang kurang efisien dan aman dari tabrakan [9], [10], [5].

Kontribusi dan inovasi utama dari penelitian ini dirangkum sebagai berikut:

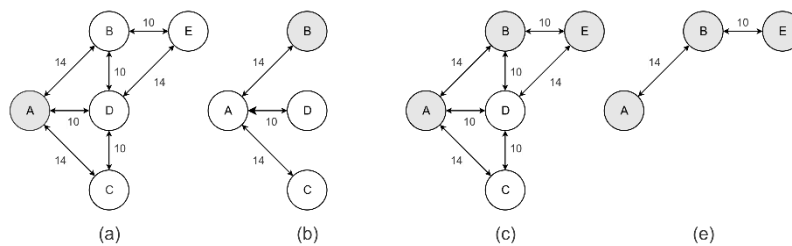
- (a) Menggabungkan *Jump Point Search* (JPS) untuk mengurangi simpul pencarian pada jalur lurus.
- (b) Mengintegrasikan *Bidirectional Search* (BDS) untuk mempercepat pencarian dari dua arah.

- (c) Menerapkan Guide Line dan BRC guna membatasi area pencarian berdasarkan kepadatan rintangan.
- (d) Mengoptimalkan jalur dengan Path Polyline Optimization (PPO) agar sesuai dengan kinematika robot.
- (e) Menyusun kombinasi metode di atas untuk meningkatkan efisiensi, kecepatan, dan keamanan jalur.

2. Algoritma A*

A^* (dibaca "*A-star*") adalah algoritma komputer yang populer dalam pathfinding dan

graph traversal, digunakan untuk mencari jalur paling efisien antar simpul (*node*) [11]. Algoritma ini terkenal karena akurasi dan kinerjanya, serta banyak diterapkan di berbagai bidang [11]. Dengan menggabungkan *Best First Search* (BFS) dan *heuristik*, A^* menemukan jalur dengan biaya terendah dari simpul awal ke simpul tujuan. Prosesnya mengandalkan fungsi yang menggabungkan jarak aktual dan estimasi jarak ke tujuan untuk menentukan urutan pencarian simpul [11].



Gambar 1 Pencarian Algoritma A^*

Algoritma A^* bekerja dengan memilih node dengan nilai $f(n)$ terendah dalam setiap iterasi, yang memungkinkan pencarian jalur optimal dengan efisiensi tinggi. Fungsi *heuristik* seperti *Euclidean distance* atau *Manhattan distance* sering digunakan untuk menghitung nilai $h(n)$, memastikan efisiensi algoritma dalam lingkungan grid [9], [12].

$$f(n) = g(n) + h(n) \quad (2.1)$$

Fungsi $f(n)$ adalah fungsi biaya jalur, dan biaya titik awal ke *node* saat ini dilambangkan

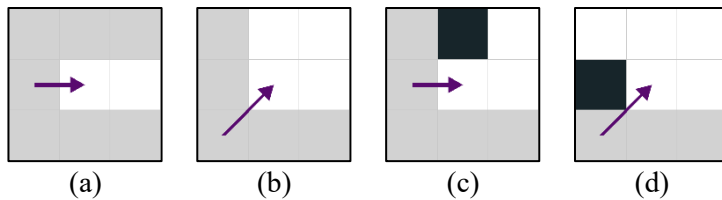
sebagai $g(n)$ dan biaya perkiraan dari titik saat ini ke tujuan dilambangkan dengan $h(n)$.

3. Peningkatan Algoritma

3.1. Jump Point Search

Jump Point Search (JPS) adalah algoritma pencarian jalur yang dirancang untuk mengurangi jumlah node yang diperiksa dalam pencarian A^* tradisional dengan cara melewati (melompat) node-node yang tidak relevan di jalur lurus [13]. JPS menggunakan pendekatan untuk mengidentifikasi node penting yang memastikan jalur tetap optimal dengan lebih sedikit langkah

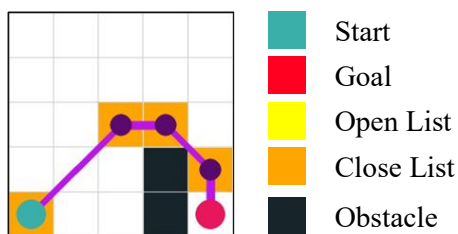
evaluasi, khususnya pada jalur lurus dan diagonal [14], [4].



Gambar 2 Aturan Tetangga Valid pada JPS

Natural neighbor *nodes* adalah node yang meluas langsung mengikuti arah pencarian dalam algoritma *Jump Point Search*. Pemanfaatan node ini memungkinkan penghindaran node yang tidak relevan secara efisien, sehingga memperkecil ruang pencarian dan mempercepat proses perencanaan jalur [14].

Forced Neighbor Nodes adalah node yang harus dipertimbangkan dalam algoritma jumping point search untuk memastikan jalur tetap optimal meskipun ada rintangan. Dengan memeriksa *node-node* ini, algoritma dapat menghindari halangan dengan lebih efektif dan memastikan jalur yang ditemukan tetap valid dan optimal [14].



Gambar 3 Pencarian Jalur Menggunakan JPS

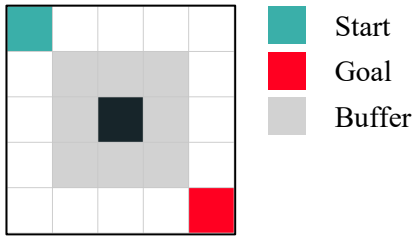
3.2. Bidirectional Search

Bidirectional Search adalah teknik pencarian yang bekerja dengan memulai proses

dari dua arah sekaligus, yakni dari titik awal dan titik tujuan [7]. Pendekatan ini bertujuan mempercepat waktu pencarian dengan mempertemukan kedua jalur di suatu titik di tengah, yang secara signifikan mengurangi jumlah simpul yang perlu diperiksa dibandingkan dengan pencarian satu arah tradisional yang hanya bergerak dari satu sisi menuju tujuan [12]. Proses ini berlangsung secara bergantian hingga kedua jalur bertemu di suatu simpul, menandakan bahwa lintasan yang dicari telah ditemukan [5].

3.3. Expansion Distance

Expansion Distance adalah jarak aman yang dijaga antara jalur robot dan hambatan selama perencanaan jalur, bertujuan untuk mengurangi risiko tabrakan. Jarak ini dihitung sebagai area buffer di sekitar hambatan, yang memastikan jalur robot tetap aman dengan memperbesar ruang di sekitar hambatan [9].



Gambar 4 Expansion Distance

Ukuran *Expansion Distance* bergantung pada faktor seperti kecepatan robot, ukuran robot, dan grid peta yang digunakan. Semakin tinggi kecepatan robot, semakin besar *Expansion Distance* yang diperlukan untuk menjaga ketahanan jalur dan mencegah tabrakan. Dengan memperbesar area ekspansi, algoritma perencanaan jalur dapat mengurangi jumlah simpul yang harus dijelajahi, sehingga meningkatkan efisiensi perencanaan jalur dan mengurangi risiko kecelakaan [9].

3.4. Barrier Raster Coefficient

Barrier Raster Coefficient adalah nilai yang digunakan dalam peta raster untuk menggambarkan tingkat hambatan atau kesulitan navigasi pada suatu area. Nilai ini digunakan dalam algoritma perencanaan jalur untuk menunjukkan seberapa sulit atau tidak dapat dilalui suatu wilayah, dengan nilai lebih tinggi biasanya menunjukkan hambatan yang lebih besar atau area yang lebih sulit dilalui oleh robot atau kendaraan [9].

$$P = \frac{N}{(|X_s - X_g| + 1) * (|y_s - y_g| + 1)} \quad (3.1)$$

Koefisien ini membantu algoritma dalam membuat keputusan mengenai jalur yang akan dipilih, dengan menghindari area yang memiliki nilai koefisien hambatan tinggi. *Barrier Raster Coefficient* penting dalam mengoptimalkan perencanaan jalur dengan memperhitungkan kondisi fisik atau lingkungan yang menghalangi perjalanan [6].

$$f(n) = g(n) + (1 - \ln P) \times h(n) \quad (3.2)$$

Fungsi evaluasi dimodifikasi di mana P merepresentasikan prioritas atau probabilitas jalur. Modifikasi ini bertujuan menyesuaikan kontribusi heuristik berdasarkan tingkat kepercayaan terhadap jalur yang dipilih.

3.5. Guide Line

Guide Line merupakan metode dalam algoritma pencarian jalur yang digunakan untuk membatasi area pencarian agar tetap fokus pada jalur langsung menuju tujuan. Metode ini bertujuan untuk meningkatkan efisiensi dengan memastikan bahwa pencarian tidak menyimpang terlalu jauh dari jalur terpendek, meskipun terdapat hambatan atau rintangan dalam grid. Dengan menggunakan *Guide Line*, algoritma dapat mengarahkan pencarian agar tetap berada di sekitar jalur yang optimal, mengurangi area pencarian yang tidak perlu, dan mempercepat waktu pencarian [5].

$$C(n) = \frac{|A \cdot x_0 + B \cdot y_0 + C|}{\sqrt{A^2 + B^2}} \quad (3.3)$$

Secara keseluruhan, *Guide Line* bertujuan untuk meningkatkan kemampuan algoritma untuk mengurangi penggunaan ruang pencarian yang tidak perlu, dan memastikan pencapaian tujuan secara lebih akurat [5].



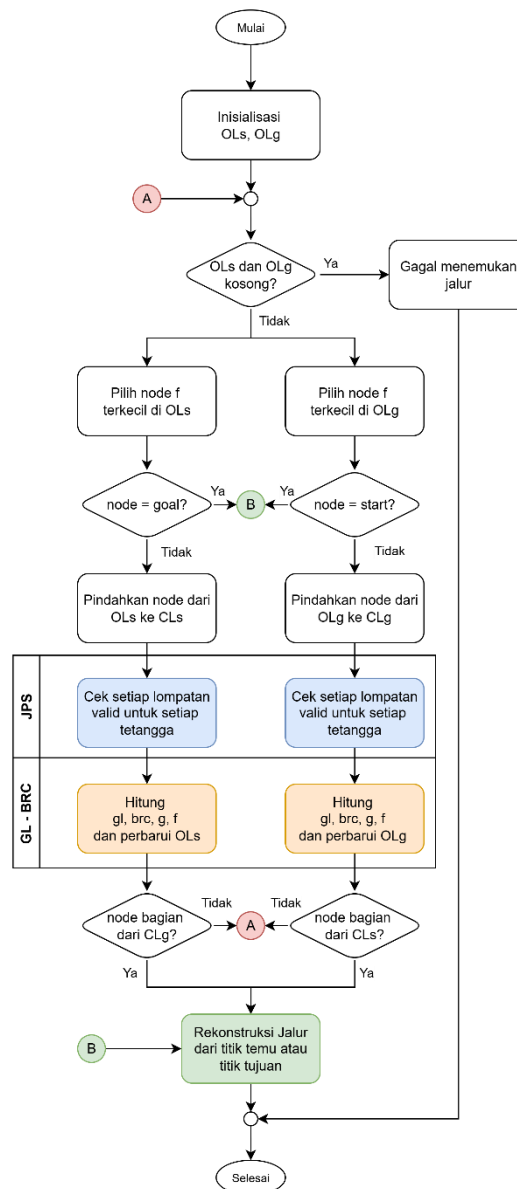
Gambar 5 Ilustrasi Path Polyline Optimization

Proses ini dimulai dengan menghapus *node* kolinear yang tidak perlu, kemudian mengeliminasi titik belok yang berlebihan. Jalur yang dioptimalkan diperoleh dengan menghubungkan *node* secara berurutan,

3.6. Path Polyline Optimization

Path Polyline Optimization adalah metode untuk mengoptimalkan jalur dengan menghilangkan titik belok *node redundant* dan menyederhanakan jalur agar lebih sesuai dengan model kinematik robot bergerak [7].

memeriksa apakah segmen garis melewati hambatan, dan jika ada, mengubah titik awal untuk melanjutkan proses hingga mencapai titik akhir [7].



4. Metodologi Penelitian

Penelitian ini bersifat eksperimental dengan menguji algoritma A* tradisional dan berbagai metode optimasi, baik secara individual maupun kombinasi, pada peta grid dengan variasi ukuran dan struktur. Setiap metode diuji menggunakan heuristik Euclidean untuk mengevaluasi kinerja

berdasarkan waktu pencarian, jumlah simpul yang dieksplorasi, dan jumlah belokan.

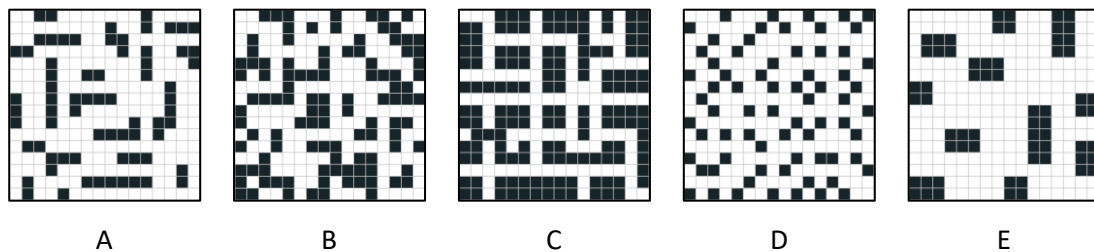
Analisis dilakukan secara kuantitatif dan visual untuk melihat karakteristik jalur, perilaku algoritma, serta respons terhadap rintangan. Sebagai validasi, hasil terbaik dari simulasi diimplementasikan pada robot mobile untuk menguji kesesuaian jalur di dunia nyata.

4.1. Design Eksperimen

Setiap metode optimasi diuji secara terpisah dengan mengintegrasikannya ke dalam algoritma A* tradisional untuk mengevaluasi pengaruh individual terhadap kinerja. Pengujian dilakukan pada kondisi peta dan ukuran grid yang sama, dengan A* tradisional sebagai baseline. Tujuannya adalah mengidentifikasi kontribusi masing-masing metode terhadap waktu pencarian, jumlah simpul yang dieksplorasi, dan jumlah belokan sebelum menguji kombinasi metode.

4.2. Jenis Map

Penelitian ini menggunakan berbagai peta raster dengan variasi pola dan kepadatan rintangan, diklasifikasikan sebagai: (a) simple map, (b) complex map, (c) structured map, (d) random map, (d) simple map. Variasi ini merepresentasikan lingkungan uji yang beragam untuk mengevaluasi kemampuan adaptif algoritma terhadap kondisi berbeda.

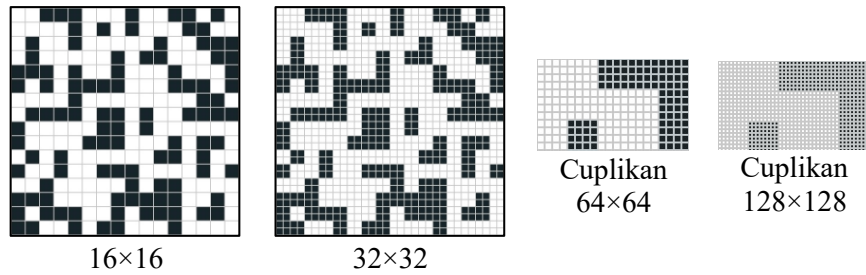


Gambar 6 Variasi Map Pengujian

Kombinasi peta yang digunakan mencakup variasi pola dan kepadatan rintangan untuk merepresentasikan kondisi lingkungan yang berbeda, seperti jalur sempit, area terbuka, serta distribusi rintangan acak dan terstruktur.

Dalam mengevaluasi konsistensi performa terhadap perubahan skala spasial, setiap jenis peta diperluas secara bertahap dari ukuran dasar 16×16 piksel hingga 128×128 piksel dalam kelipatan dua.

Meskipun pola rintangan pada peta dipertahankan, peningkatan skala digunakan untuk merepresentasikan kebutuhan perencanaan jalur yang lebih presisi. Pendekatan ini memungkinkan analisis performa algoritma pada peta berukuran besar serta responsnya terhadap kompleksitas spasial yang meningkat, khususnya terkait ketelitian jalur dan efisiensi komputasi.



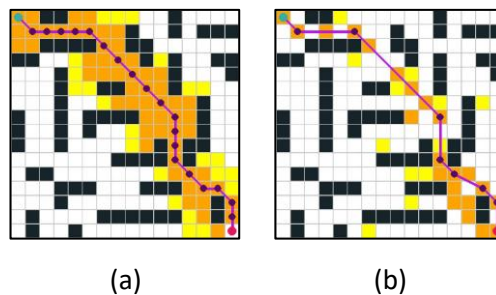
Gambar 7 Variasi Ukuran Map

5. Hasil dan Analisis

Evaluasi algoritma A^* yang dioptimasi dilakukan melalui pengujian pada lima peta uji berukuran 16×16 hingga 128×128 grid untuk menganalisis skalabilitas. Nilai parameter K ditetapkan sebesar 0,5 sebagai konstanta penalti untuk setiap manuver belokan, mengingat penelitian sebelumnya [4] tidak menyebutkan nilai ini secara eksplisit. Kinerja algoritma yang diusulkan dibandingkan dengan algoritma A^*

konvensional sebagai baseline, mencakup parameter waktu komputasi, jumlah node yang dieksplorasi, dan jumlah belokan pada jalur akhir.

Seperti yang ditunjukkan pada Gambar 32, terdapat perbandingan antara jalur yang dihasilkan oleh algoritma A^* tradisional dengan hasil algoritma yang telah dioptimasi dalam penelitian ini

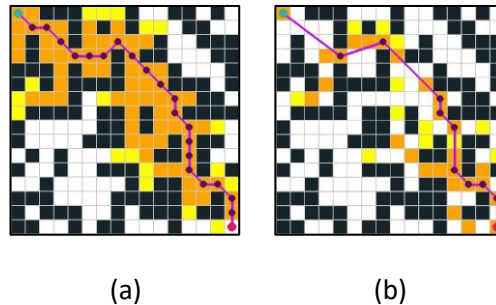


Gambar 8 Hasil Jalur pada Peta A ukuran 16×16

Map	Ukuran Map	Algoritma	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Jalur
A	16 x 16	A^*	88	7	0,0004638	24,142
		Improved A^*	37	5	0,0004754	23,604
A	32 x 32	A^*	317	12	0,0033869	49,698
		Improved A^*	54	5	0,0013899	48,097
A	64 x 64	A^*	1149	14	0,0240776	100,811
		Improved A^*	56	5	0,0045273	97,221
A	128 x 128	A^*	4358	22	0,1518754	203,037
		Improved A^*	57	5	0,0179984	195,518

Pada peta dengan distribusi rintangan merata dan ruang terbuka untuk loncatan yang memadai, Map A menunjukkan peningkatan performa yang signifikan di segala aspek,

termasuk penurunan panjang jalur yang dihasilkan.

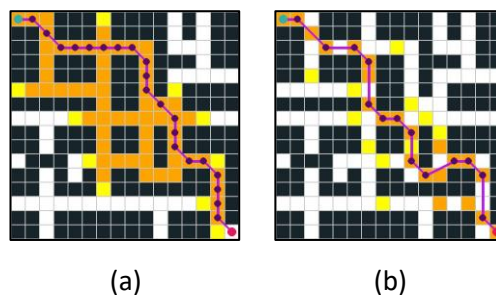


Gambar 9 Gambar 30 Hasil Jalur pada Peta B ukuran 16 x 16

Map	Ukuran Map	Algoritma	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Jalur
B	16 x 16	A^*	91	12	0,0004757	25,556
		Improved A^*	36	6	0,0004256	24,837
B	32 x 32	A^*	321	12	0,0028988	51,698
		Improved A^*	65	7	0,0014189	51,455
B	64 x 64	A^*	1197	18	0,0186209	103,983
		Improved A^*	62	7	0,0037633	103,461
B	128 x 128	A^*	4515	32	0,1221172	208,551
		Improved A^*	60	8	0,0128954	207,749

Pada Map B, algoritma yang optimasi menunjukkan keunggulan dalam sebagian besar parameter kinerja, namun mengalami *trade-off* pada aspek panjang jalur dengan terjadinya peningkatan panjang jalur pada solusi akhir.

Pada peta terstruktur yang ditunjukkan pada Gambar 34 menunjukkan perbedaan dalam pemilihan jalur dengan A^* tradisional.



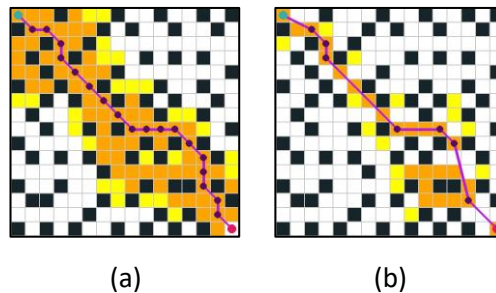
Gambar 10 Hasil Jalur pada Peta C ukuran 16 x 16

Map	Ukuran Map	Algoritma	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Jalur
C	16 x 16	A^*	72	11	0,0003802	25,314
		<i>Improved A*</i>	34	11	0,0003581	26,075
C	32 x 32	A^*	288	16	0,0024911	52,042
		<i>Improved A*</i>	61	13	0,0011095	53,447
C	64 x 64	A^*	1104	22	0,0158681	105,497
		<i>Improved A*</i>	54	13	0,0030103	106,820
C	128 x 128	A^*	4185	26	0,1031649	212,409
		<i>Improved A*</i>	54	13	0,010972	214,321

Pada Map A dan Map B, algoritma optimasi menunjukkan peningkatan performa pada semua aspek. Namun, berbeda dengan Map C, terjadi perbedaan dalam pemilihan lintasan oleh algoritma optimasi yang justru menyebabkan peningkatan panjang jalur. Hal ini menunjukkan bahwa karakteristik topologi pada Map C mempengaruhi efektivitas

algoritma, sehingga lintasan yang dihasilkan menjadi lebih panjang dan kurang efisien dibandingkan dengan algoritma baseline.

Peningkatan performa kembali terjadi pada peta selanjutnya yaitu pada Map D yang ditunjukkan oleh Gambar 35 di bawah ini.



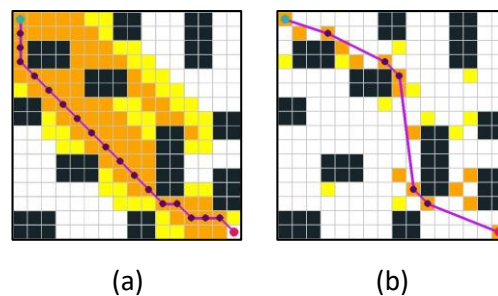
Gambar 11 Hasil Jalur pada Peta D ukuran 16 x 16

Map	Ukuran Map	Algoritma	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Jalur
D	16 x 16	A^*	104	10	0,0005351	23,556
		<i>Improved A*</i>	43	5	0,0004423	22,621
D	32 x 32	A^*	390	14	0,0036208	48,527
		<i>Improved A*</i>	60	6	0,0010345	46,752
D	64 x 64	A^*	1511	20	0,0287961	98,468
		<i>Improved A*</i>	60	6	0,0030974	94,901
D	128 x 128	A^*	5717	28	0,2103159	198,350
		<i>Improved A*</i>	61	8	0,0111757	197,725

Pada jenis peta dengan sebaran rintangan yang acak dan merata serta jumlah area terbuka yang cukup tinggi, algoritma yang dioptimasi menunjukkan peningkatan performa yang sangat signifikan dibandingkan variasi peta sebelumnya. Perbaikan kinerja tercermin melalui penurunan secara signifikan pada jumlah *node* dan belokan yang dihasilkan, serta

terjadi penurunan panjang jalur akhir, mengindikasikan bahwa karakteristik *topologi* dengan ruang eksplorasi yang lebih terbuka memungkinkan algoritma optimasi bekerja lebih optimal.

Terakhir hasil pada peta dengan ruang terbuka yang lebih luas ditunjukan pada Gambar 36.



Gambar 12 Hasil Jalur pada Peta E ukuran 16 x 16

Map	Ukuran Map	Algoritma	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Jalur
E	16 x 16	<i>A*</i>	150	5	0,0007653	22,971
		<i>Improved A*</i>	28	4	0,0004543	23,855
E	32 x 32	<i>A*</i>	548	12	0,0055339	47,355
		<i>Improved A*</i>	43	4	0,0020277	47,715
E	64 x 64	<i>A*</i>	1782	24	0,0473926	96,125
		<i>Improved A*</i>	43	4	0,0066388	95,510
E	128 x 128	<i>A*</i>	6183	42	0,320457	193,664
		<i>Improved A*</i>	43	4	0,0244447	192,292

Serupa dengan Map D, pada Map E ini juga menunjukkan peningkatan yang signifikan pada performa algoritma optimasi. Hal ini terlihat dari reduksi jumlah simpul yang drastis, penurunan jumlah belokan secara konsisten pada seluruh ukuran map, serta percepatan waktu pencarian yang

signifikan dibandingkan dengan algoritma *A** tradisional. Namun, algoritma optimasi tetap mengalami peningkatan panjang jalur akhir yang relatif kecil, yang masih dalam rentang yang dapat diterima mengingat peningkatan substansial pada parameter-parameter lainnya.

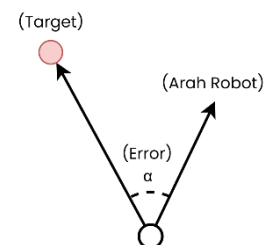
Table 1 Perbandingan Peningkatan Performa Pada Setiap Map

Peningkatan Performa					
Map	Ukuran Map	Jumlah Simpul	Jumlah Belokan	Waktu	Panjang Lintasan
A	16 x 16	57,95%	28,57%	-2,50%	2,23%
	32 x 32	82,97%	58,33%	58,96%	3,22%
	64 x 64	95,13%	64,29%	81,20%	3,56%
	128 x 128	98,69%	77,27%	88,15%	3,70%
B	16 x 16	60,44%	50,00%	10,53%	2,81%
	32 x 32	79,75%	41,67%	51,05%	0,47%
	64 x 64	94,82%	61,11%	79,79%	0,50%
	128 x 128	98,67%	75,00%	89,44%	0,38%
C	16 x 16	52,78%	0,00%	5,81%	-3,01%
	32 x 32	78,82%	18,75%	55,46%	-2,70%
	64 x 64	95,11%	40,91%	81,03%	-1,25%
	128 x 128	98,71%	50,00%	89,36%	-0,90%
D	16 x 16	58,65%	50,00%	17,34%	3,97%
	32 x 32	84,62%	57,14%	71,43%	3,66%
	64 x 64	96,03%	70,00%	89,24%	3,62%
	128 x 128	97,02%	64,29%	94,91%	-0,42%
E	16 x 16	83,56%	0,00%	37,81%	-4,09%
	32 x 32	93,25%	66,67%	62,33%	-0,76%
	64 x 64	96,89%	79,17%	85,73%	0,62%
	128 x 128	98,59%	88,10%	92,38%	0,70%

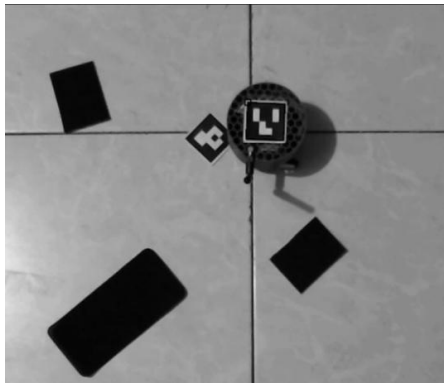
6. Eksperimen Validasi

Dalam memverifikasi kelayakan algoritma A* yang telah ditingkatkan dalam penerapan nyata pada robot mobile, algoritma ini diterapkan pada robot diferensial sederhana beroda dua yang dikendalikan melalui server, sebagaimana ditunjukkan pada Gambar 14a. Sistem ini memanfaatkan kamera overhead untuk melakukan pengamatan lingkungan dari atas. Informasi visual yang diperoleh digunakan untuk pelokalan robot dan membangun peta dua

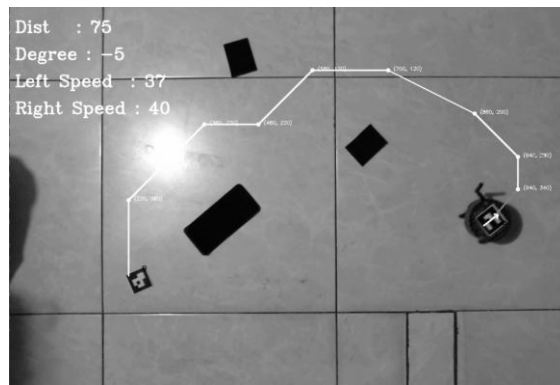
dimensi area uji, dengan posisi robot dibaca secara real-time menggunakan penanda *ArUco*. Perencanaan jalur global dijalankan menggunakan algoritma A* yang telah dimodifikasi.



Gambar 13 Penentuan error arah robot



Lingkungan Arena



Pembentukan Jalur dan Navigasi

Gambar 14 Simulasi pada lingkungan nyata

Algoritma A* yang dioptimasi mampu menemukan jalur secara efisien dengan waktu komputasi yang lebih cepat dibanding versi standar. Implementasi konsep expansion distance menghasilkan margin aman antara jalur dan rintangan, sehingga meningkatkan keselamatan navigasi dan mencegah potensi tabrakan dengan objek di lingkungan

7. Kesimpulan

Berdasarkan hasil evaluasi, optimasi terhadap algoritma A* menunjukkan peningkatan performa yang signifikan dibandingkan dengan algoritma A* tradisional. Jumlah simpul yang dievaluasi berkurang sebesar 85,12%, menunjukkan efisiensi pencarian jalur yang lebih baik. Jumlah belokan dalam lintasan juga berhasil ditekan hingga 52,06%, yang berkontribusi terhadap kelancaran navigasi. Waktu pencarian mengalami peningkatan efisiensi

sebesar 61,97%, menandakan bahwa algoritma yang dioptimasi mampu menemukan jalur lebih cepat. Meskipun panjang lintasan hanya berkurang sebesar 0,82%, hal ini tidak berdampak signifikan terhadap kualitas jalur secara keseluruhan.

Implementasi algoritma pada robot diferensial beroda dua yang dikendalikan melalui server membuktikan keberhasilan penerapan di lingkungan nyata. Sistem menggunakan kamera overhead dan penanda ArUco untuk pelokalan visual, serta menjalankan jalur hasil perencanaan algoritma A* yang telah dioptimasi. Robot mampu mengikuti lintasan yang ditentukan secara akurat dan aman tanpa mengalami tabrakan, berkat penerapan konsep expansion distance yang memberikan margin aman terhadap rintangan di sekitarnya.

Referensi

- [1] Z. Yang, J. Li, L. Yang, dan H. Chen, "A Smooth Jump Point Search Algorithm for Mobile Robots Path Planning Based on a Two-Dimensional Grid Model," *Journal of Robotics*, vol. 2022, 2022, doi: 10.1155/2022/7682201.
- [2] C. Yin, C. Tan, C. Wang, dan F. Shen, "An Improved A-Star Path Planning Algorithm Based on Mobile Robots in Medical Testing Laboratories," *Sensors*, vol. 24, no. 6, Mar 2024, doi: 10.3390/s24061784.
- [3] D. Harabor dan A. Grastien, "Online Graph Pruning for Pathfinding on Grid Maps." [Daring]. Tersedia pada: www.aai.org
- [4] H. Gong *dkk.*, "Bidirectional Jump Point Search Path-Planning Algorithm Based on Electricity-Guided Navigation Behavior of Electric Eels and Map Preprocessing," *Biomimetics*, vol. 8, no. 5, Sep 2023, doi: 10.3390/biomimetics8050387.
- [5] T. XiangRong, Z. Yukun, dan J. XinXin, "Improved a-star algorithm for robot path planning in static environment," dalam *Journal of Physics: Conference Series*, IOP Publishing Ltd, Feb 2021. doi: 10.1088/1742-6596/1792/1/012067.
- [6] L. Chu *dkk.*, "Intelligent Vehicle Path Planning Based on Optimized A* Algorithm," *Sensors*, vol. 24, no. 10, Mei 2024, doi: 10.3390/s24103149.
- [7] P. Li, Y. Li, dan X. Dai, "VNS-BA*: An Improved Bidirectional A* Path Planning Algorithm Based on Variable Neighborhood Search," *Sensors*, vol. 24, no. 21, hlm. 6929, Okt 2024, doi: 10.3390/s24216929.
- [8] N. Liu, C. Ma, Z. Hu, P. Guo, Y. Ge, dan M. Tian, "Workshop AGV path planning based on improved A* algorithm," *Mathematical Biosciences and Engineering*, vol. 21, no. 2, hlm. 2137–2162, 2024, doi: 10.3934/mbe.2024094.
- [9] H. Wang, S. Lou, J. Jing, Y. Wang, W. Liu, dan T. Liu, "The EBS-A* algorithm: An improved A* algorithm for path planning," *PLoS One*, vol. 17, no. 2 February, Feb 2022, doi: 10.1371/journal.pone.0263841.
- [10] J. Chen, C. Tan, R. Mo, H. Zhang, G. Cai, dan H. Li, "Research on path planning of three-neighbor search A* algorithm combined with artificial potential field," *Int J Adv Robot Syst*, vol. 18, no. 3, 2021, doi: 10.1177/17298814211026449.

- [11] I. Bagus, G. Wahyu, dan A. Dalem, “PENERAPAN ALGORITMA A* (STAR) MENGGUNAKAN GRAPH UNTUK MENGHITUNG JARAK TERPENDEK,” Online, 2018. [Daring]. Tersedia pada: <http://jurnal.stiki-indonesia.ac.id/index.php/jurnalresistor>
- [12] Y. Sun, Q. Yuan, Q. Gao, dan L. Xu, “A Multiple Environment Available Path Planning Based on an Improved A* Algorithm,” *International Journal of Computational Intelligence Systems*, vol. 17, no. 1, Des 2024, doi: 10.1007/s44196-024-00571-z.
- [13] J. G. Min, W. S. Ruy, dan C. S. Park, “Faster pipe auto-routing using improved jump point search,” *International Journal of Naval Architecture and Ocean Engineering*, vol. 12, hlm. 596–604, Jan 2020, doi: 10.1016/j.ijnaoe.2020.07.004.
- [14] F. Wang, W. Sun, P. Yan, H. Wei, dan H. Lu, “Research on Path Planning for Robots with Improved A* Algorithm under Bidirectional JPS Strategy,” *Applied Sciences (Switzerland)*, vol. 14, no. 13, Jul 2024, doi: 10.3390/app14135622.